

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

File Operations Using Node.js File System (fs) and Readline Modules

2. AIM

To design and develop an interactive command-line Node.js application to perform fundamental file handling operations including creating, writing, reading, and appending data to files using the built-in fs and readline modules.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and use core Node.js modules (fs and readline) for system operations.
 - To capture terminal user input interactively using readline.createInterface() and rl.question().
 - To write data to a file synchronously using fs.writeFileSync().
 - To read file contents synchronously with UTF-8 encoding using fs.readFileSync().
 - To append additional content to an existing file using fs.appendFileSync().
 - To properly close the I/O stream using rl.close().
-

4. THEORY

Node.js File System Module (fs) — The built-in fs module provides APIs for interacting with the file system on the host operating system.

Synchronous File Operations — Methods such as writeFileSync(), readFileSync(), and appendFileSync() block execution until the I/O operation finishes, ensuring sequential file processing.

fs.writeFileSync(filename, data) — Synchronously creates a new file or overwrites an existing file with the specified text content.

fs.readFileSync(filename, "utf8") — Synchronously reads and returns the complete content of a file decoded into a UTF-8 character string.

fs.appendFileSync(filename, data) — Appends text data to the end of an existing file without truncating prior content.

readline Module — Provides an interface for reading data line-by-line from a readable stream such as process.stdin.

5. ALGORITHM / PROCEDURE

- Create a JavaScript file named fileOperations.js.
 - Import the built-in fs and readline modules using require().
 - Initialize the readline interface configured with process.stdin and process.stdout.
 - Prompt the user for a filename using rl.question().
 - Prompt the user for initial file content.
 - Write the content to the specified file using fs.writeFileSync(filename, content).
 - Read the initial file content using fs.readFileSync(filename, "utf8") and display it on the console.
 - Prompt the user to enter additional content.
 - Append the new content preceded by a newline character using fs.appendFileSync(filename, "\n" + additional).
 - Read the file again and display the updated final content on the console.
 - Close the readline interface using rl.close().
 - Execute the program using node fileOperations.js and record test results.
-

6. PROGRAM

file-manager.js

```
const fs = require("fs");
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question("Enter filename: ", (filename) => {
```

```

    rl.question("Enter content: ", (content) => {

        // 1. Create/Write the file
        fs.writeFileSync(filename, content);

        // 2. Read the file
        console.log("\nInitial Content:");
        console.log(fs.readFileSync(filename, "utf8"));

        // 3. Append additional content
        rl.question("\nEnter additional content: ", (additional) =>
        {
            fs.appendFileSync(filename, "\n" + additional);

            // 4. Display final contents
            console.log("\nFinal Content:");
            console.log(fs.readFileSync(filename, "utf8"));

            rl.close();
        });
    });
});

```

7. CODE EXPLANATION

Code	Explanation
<code>require("fs")</code>	Imports the core Node.js File System module for file manipulation.
<code>readline.createInterface({...})</code>	Creates a readline interface instance bound to standard input and output streams.
<code>rl.question(prompt, callback)</code>	Displays the prompt string and executes the callback with user input.
<code>fs.writeFileSync(filename, content)</code>	Synchronously writes the initial content to the specified file.
<code>fs.readFileSync(filename, "utf8")</code>	Synchronously reads the file and returns its content as a UTF-8 decoded string.
<code>fs.appendFileSync(filename, "\n" + additional)</code>	Appends additional text to the end of the file on a new line.
<code>rl.close()</code>	Closes the readline interface and terminates input listening.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Provide filename sampleFile.txt and content "Hello world!"	Creates sample.txt and prints "Initial Content: Hello world!"	File created and initial content printed	Pass
TC-02	Provide additional content "Welcome to FS Module Demo"	Appends "Welcome to FS Module Demo" and prints combined content	Final content displays both lines	Pass

9. OUTPUT

Output 1: Create sampleFile.txt and write content

Creates sample.txt and prints "Initial Content: Hello world!".

```
Problems  Debug Console  Output  Terminal  Ports
○ PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_5\PBL-2> node file-manager.js
Enter filename: sampleFile
Enter content: Hello world!

Initial Content:
Hello world!

Enter additional content: 
```

Output 2: Provide additional content

Creates sample.txt and prints "Initial Content: Hello world!".

```
Enter additional content: welcome to FS Module Demo

Final Content:
Hello world!
welcome to FS Module Demo
○ PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_5\PBL-2> 
```

10. RESULT

Thus, an interactive command-line Node.js application was successfully developed to perform file handling operations using the fs and readline modules. The program demonstrates file creation, synchronous writing, reading, and appending data. All test cases passed successfully.